ASSIGNMENT – 2
CRYPTOGRAPHY

EVALUATE, DESIGN AND DEFEND A SECURE PUBLIC-KEY AND HYBRID CRYPTOGRAPHIC STRATEGY FOR A MULTI-SITE HEALTHCARE NETWORK

Submitted by: T. Subahan

Register Number: 192511121

Department: BE CSE

Course: Cryptography and network security

Date: 11/11/26

Declaration

I declare that this report is prepared for academic learning. The calculations use small toy parameters and fictional test data so that the steps can be checked by hand. Toy cryptography is not suitable for protecting real patient information. Any execution-time values must be measured on the student's own computer; this report provides reproducible procedures and clearly labels illustrative calculations.

Acknowledgement

I thank the course faculty and institution for the opportunity to study symmetric and public-key cryptography, evaluate their limitations, and apply mathematical reasoning to a healthcare communication scenario.

Abstract

This assignment evaluates symmetric encryption, block-cipher modes, number-theory foundations, RSA, Diffie–Hellman key exchange, elliptic-curve cryptography, digital signatures, hybrid encryption and quantum-readiness. The scenario is a healthcare network in which a central hospital exchanges clinical records with remote clinics over public networks. The report follows the assignment's sequence: compare algorithms, implement mathematical building blocks, examine attacks, design authenticated key exchange, compare public-key choices, construct a hybrid architecture, and redesign it under a larger-scale threat. The central engineering conclusion is that public-key cryptography should establish or protect short-lived symmetric keys, while authenticated modern symmetric encryption protects bulk data. Authentication, key management, secure randomness, implementation quality and migration planning are as important as the mathematical algorithm.

Contents and assignment alignment

Question	Main coverage	Marks
Q1	DES, 3DES, Blowfish, RC5; block/stream ciphers; ECB, CBC, CFB, OFB, CTR	7
Q2	GCD, extended Euclid, modular inverse, modular exponentiation, primality tests	6
Q3	RSA, textbook weaknesses, factoring, padding and OAEP	7
Q4	Diffie–Hellman, MITM, signatures/certificates and key management	7
Q5	ECC, toy example and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid	7

	encryption, tamper tests and quantum readiness	
Q7	Integrated diagnosis, redesign, validation and defence	11

Scenario, scope and assumptions

A central hospital and a group of remote clinics exchange patient records, prescriptions, diagnostic reports and billing information over public networks. The system must keep stored and transmitted clinical records confidential, establish session keys without relying on an observable network being private, authenticate communicating parties, detect modification, support large files efficiently and remain usable on constrained clinic equipment. The design must also prepare for future changes in cryptographic risk.

The assignment asks for comparisons and student-created demonstrations. Accordingly, examples use small integers, short strings and fictional records. Such examples demonstrate the arithmetic but are deliberately insecure. A real deployment must use reviewed cryptographic libraries, current standards, secure random-number generation, protected key storage, authenticated protocols and operational monitoring.

Notation and working assumptions

- For RSA, $n = p \times q$ and $\varphi(n) = (p - 1)(q - 1)$ when p and q are distinct primes.
- For Diffie–Hellman, p is a prime modulus, g is a generator or suitable base, and private exponents are secret.
- For ECC, point addition and scalar multiplication are performed on a selected, validated curve; toy curves are only for arithmetic illustration.
- Execution time depends on processor, programming language, library, operating system and measurement method. Do not present illustrative timings as measured results.
- A nonce or IV is not necessarily secret. Its uniqueness or unpredictability requirement depends on the mode and protocol.
- All example patient names, identifiers and records are fictional. No real patient data should be used in classroom experiments.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Objective and interpretation of the question

The objective is to compare the assigned symmetric algorithms and determine which construction is appropriate for clinical records. The question is not answered by choosing the algorithm with the largest apparent key space. Block size, key length, known cryptanalytic weaknesses, mode of operation, implementation availability and operational support all affect the decision.

Core concepts and technical explanation

A symmetric cipher uses the same secret key, or closely related secret key material, for encryption and decryption. The sender and receiver must therefore obtain the key securely before protected communication can begin. Symmetric algorithms are efficient for large amounts of data, which makes them suitable for files and database records once a session key has been established.

A block cipher transforms a fixed-size block under a key. DES operates on 64-bit blocks and historically uses a 56-bit effective key. Its Feistel structure has 16 rounds. In each round, one half is combined with a round-key-dependent function and XORed with the other half. The short effective key makes exhaustive search practical with modern resources; DES is a legacy algorithm and should not protect new data.

Triple DES applies DES three times, commonly in an encrypt-decrypt-encrypt sequence. It was introduced to extend DES security while reusing DES hardware and software. Its 64-bit block size remains a serious limitation for large volumes, and its performance is slower than modern designs. It is a legacy compatibility option, not a preferred new system.

Blowfish is a 16-round Feistel block cipher with a 64-bit block and variable key length. It uses key-dependent substitution tables. Its key setup is relatively expensive, while repeated encryption after setup can be efficient. The 64-bit block size is a concern when encrypting large data volumes under one key. RC5 is a parameterised block cipher: block size, key size and number of rounds are selectable. It uses data-dependent rotations, XOR and modular addition. Flexibility is useful for study, but security depends on the chosen parameters and implementation.

A stream cipher generates a pseudorandom keystream and combines it with plaintext, usually by XOR. The same keystream must never be reused with different plaintext under the same key. Reuse can reveal relationships between plaintexts. Stream constructions are useful for continuous data streams and low-latency communication, while block ciphers are natural for fixed-size blocks and bulk storage. Modern authenticated encryption modes can provide both confidentiality and integrity.

Block-cipher modes specify how a block cipher processes messages longer than one block. ECB encrypts each block independently and reveals repeated-block patterns. CBC chains each plaintext block with the preceding ciphertext and requires a suitable unpredictable IV. CFB and OFB turn a block cipher into stream-like operation. CTR encrypts counter values to create a keystream and supports parallel processing, but counter/nonce reuse is catastrophic. Modern systems commonly prefer authenticated modes such as AES-GCM or ChaCha20-Poly1305 rather than using unauthenticated encryption alone.

Design, worked reasoning and evidence

Comparison table: DES has a 64-bit block and 56-bit effective key; 3DES retains a 64-bit block and improves resistance to exhaustive key search at a substantial performance cost; Blowfish has a 64-bit block and variable key size; RC5 is configurable and must be evaluated at the exact parameter set. A report should record the actual library parameters used rather than simply writing 'RC5'.

Example of ECB pattern leakage: imagine a test image or document containing repeated identical blocks. ECB maps equal plaintext blocks under one key to equal ciphertext blocks. An observer

cannot directly read the block contents, but can see that the same block pattern repeats and may infer structure. This is why ECB is unsuitable for structured medical images and repeated records.

Example of CBC chaining: let P1 and P2 be two plaintext blocks, C0 the IV, and E the block encryption function. C1 = E(P1 XOR C0); C2 = E(P2 XOR C1). A change in P1 affects C1 and then changes the input to the next block. CBC needs padding for non-block-aligned messages and does not itself authenticate the message.

Example of CTR: for counter blocks T1, T2, …, compute stream blocks S_i = E_K(T_i), then C_i = P_i XOR S_i. Decryption repeats the same operation. The counter values must be unique for a given key. CTR permits parallel block processing and random access, but unauthenticated CTR encryption is malleable: flipping a ciphertext bit flips the corresponding plaintext bit.

Reproducible student test: create a fixed fictional message such as 'CLINIC REPORT SAMPLE 001'. Pad or frame it as required, encrypt with each selected algorithm using documented test keys, decrypt, and assert that the recovered bytes equal the original bytes. Repeat with a second message containing repeated blocks. Record ciphertext length, number of blocks, setup time, encryption time and decryption time. Use the same machine, language, library version and input files for all runs.

Illustrative comparison of modes: ECB is parallel and simple but leaks repeated patterns; CBC hides equal plaintext blocks when IVs are correctly chosen but encryption is sequential; CFB supports stream-like processing but has feedback and error-propagation behaviour; OFB produces a keystream and ciphertext bit errors do not spread through later plaintext blocks, but the IV/nonce must not repeat; CTR is parallel and efficient but requires unique counter blocks and authentication. The exact error behaviour should be verified for the implementation and mode variant used.

Evaluation, limitations and deployment implications

Legacy algorithms can remain in old systems because of compatibility, but new deployments should not select DES or 3DES for fresh patient data. Blowfish's 64-bit block size and RC5's parameter sensitivity must be considered. A strong key size cannot compensate for a weak mode, repeated nonce, poor random generator or exposed key.

For the healthcare scenario, the recommended direction is a modern authenticated symmetric construction for bulk data. If the assignment requires only the listed algorithms, implement at least three for comparison and clearly separate that academic comparison from the real deployment recommendation. Never imply that classroom DES, Blowfish or toy RC5 tests make patient records safe.

Confidentiality without integrity is insufficient for prescriptions. A ciphertext bit may be modified in a malleable mode, potentially changing a decrypted value. Authentication tags or a separate digital-signature mechanism are required to detect modification and authenticate the sender.

Question conclusion

Q1 demonstrates that algorithm choice and mode choice must be evaluated together. The legacy algorithms are valuable for understanding block-cipher design, but a practical healthcare network should use a current authenticated-encryption standard, unique nonces, secure key generation and managed key rotation.

Algorithm	Structure / block size	Key characteristics	Evaluation for this scenario
DES	16-round Feistel; 64-bit block	56-bit effective key	Obsolete; brute-force resistance is inadequate.
3DES	DES applied three times; 64-bit block	Two-key or three-key variants	Legacy compatibility only; slow and small block size.
Blowfish	16-round Feistel; 64-bit block	Variable key length; costly setup	Historically useful; 64-bit block is limiting for large traffic.
RC5	Parameterised block cipher	Configurable word size, rounds and key length	Must state exact parameters; flexibility is not proof of security.

Mode	Main idea	IV/nonce rule	Important property / risk
ECB	Encrypt blocks independently	No IV	Leaks repeated-block patterns.
CBC	XOR with previous ciphertext before encryption	Unpredictable, unique IV recommended	Sequential encryption; padding; no built-in authentication.
CFB	Ciphertext feedback creates stream-like operation	Unique/unpredictable IV as specified	Bit errors affect current and following data according to segment size.
OFB	Encrypt feedback state to generate keystream	Never repeat IV with same key	Ciphertext bit errors do not propagate; keystream reuse is dangerous.
CTR	Encrypt counter values to create keystream	Counter/nonce unique per key	Parallel and fast; reuse reveals plaintext relationships; malleable without tag.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Objective and interpretation of the question

This question develops the arithmetic used by RSA. A correct RSA implementation depends on modular arithmetic, modular inverses, efficient modular exponentiation and reliable prime generation. The examples below use small numbers so every step can be checked manually; they are not secure RSA parameters.

Core concepts and technical explanation

The greatest common divisor gcd(a,b) is the largest positive integer that divides both a and b. Euclid's algorithm repeatedly replaces (a,b) with (b, a mod b) until the second value becomes zero. The remaining non-zero value is the GCD. The method is efficient because the remainder decreases rapidly.

The extended Euclidean algorithm also finds integers x and y such that ax + by = gcd(a,b). If gcd(a,m)=1, then ax + my = 1. Taking both sides modulo m gives ax ≡ 1 (mod m), so x is the modular inverse of a modulo m. If the GCD is not one, the inverse does not exist.

Fast modular exponentiation calculates a^e mod n without computing the enormous integer a^e directly. Square-and-multiply repeatedly squares the base modulo n and multiplies it into the result when the current exponent bit is one. The number of multiplications grows approximately with the number of exponent bits rather than with the exponent's numeric value.

Euler's totient φ(n) counts integers from 1 through n that are relatively prime to n. For RSA with n=pq and distinct primes p and q, φ(n)=(p−1)(q−1). Select e so gcd(e,φ(n))=1, then compute d as the inverse of e modulo φ(n). The public key is (n,e), and the private exponent d must remain secret.

Primality testing is needed to select p and q. Trial division checks candidate divisors up to the square root and is simple but slow for large values. Fermat's test uses a^(n−1) ≡ 1 mod n for selected bases; some composite numbers, including Carmichael numbers, can pass it. Miller–Rabin is a probabilistic test with strong practical performance when repeated with independent bases, but a test result should be described as a probability-based assurance rather than a mathematical proof.

Design, worked reasoning and evidence

Worked GCD example: gcd(252,198). Divide 252 by 198 to get remainder 54. Divide 198 by 54 to get remainder 36. Divide 54 by 36 to get remainder 18. Divide 36 by 18 to get remainder 0. Therefore gcd(252,198)=18.

Worked modular inverse: find the inverse of 7 modulo 40. Euclid gives 40=5×7+5; 7=1×5+2; 5=2×2+1. Back-substitute: 1=5−2×2=5−2(7−5)=3×5−2×7=3(40−5×7)−2×7=3×40−17×7. Thus −17 is an inverse of 7 mod 40, equivalent to 23. Check: 7×23=161 and 161 mod 40=1.

Worked square-and-multiply example: compute 7^13 mod 33. Since 13=8+4+1, calculate 7^2=49≡16; 7^4≡16^2=256≡25; 7^8≡25^2=625≡31 (mod 33). Multiply selected powers: 31×25×7=5425; 5425 mod 33=7. Hence 7^13 mod 33=7. A program should obtain the same result.

RSA toy key example: choose p=5 and q=11. Then n=55 and φ(n)=40. Choose e=3 because gcd(3,40)=1. The inverse of 3 modulo 40 is d=27 because 3×27=81≡1 mod 40. Public key is (55,3), private exponent is 27. This key is intentionally tiny and can be factored immediately.

Prime-candidate comparison: test small candidates and include 561, 1105 and 1729. These are Carmichael numbers: they are composite but can satisfy Fermat congruences for bases relatively prime to the candidate. The experiment should record the candidate, method, selected base(s), outcome and whether the result is a false acceptance. Miller–Rabin's decomposition n−1=2^s×d and repeated squaring should be shown for at least one candidate.

Complexity comparison: trial division performs a number of trial checks that grows rapidly with candidate size. Extended Euclid uses a sequence of divisions and is far more efficient than checking every possible inverse. Square-and-multiply needs O(log e) modular multiplication steps. Naive repeated multiplication uses approximately e multiplications, making it impractical for large exponents.

Evaluation, limitations and deployment implications

Prime density explains why a search may test many odd candidates before finding a prime. For large x, the probability that a randomly selected integer near x is prime is approximately 1/ln(x). This is a broad approximation, not a guarantee for an individual candidate. Practical prime generation also imposes bit-length, top-bit, oddness and cryptographic randomness requirements.

Classroom-sized RSA can be factored by trial division because n is tiny. This does not demonstrate that a 2048-bit modulus is similarly easy. The experiment demonstrates the mechanism and scaling problem, not a break of real RSA. Avoid extrapolating a toy timing linearly to production key sizes.

Modern RSA engineering generally requires carefully generated large primes, approved public-exponent choices, side-channel-resistant implementations and validated libraries. A nominal modulus length does not rescue a weak random generator, shared primes, predictable prime generation or leaked private exponent.

Question conclusion

Q2 establishes the mathematical engine required for RSA. The modular inverse exists only when the relevant values are coprime, square-and-multiply is essential for efficient exponentiation, and primality testing must be chosen with its limitations understood.

Operation	Purpose	Expected method	Check
GCD	Test coprimality	Euclid remainder loop	Final non-zero remainder.
Modular inverse	Find x where ax ≡ 1 mod m	Extended Euclid	(a×x) mod m = 1.
Modular exponentiation	Compute a^e mod n	Square-and-multiply	Compare with small direct calculation.
Prime screening	Reject composites / identify probable primes	Trial division, Fermat, Miller–Rabin	Include Carmichael test cases.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Objective and interpretation of the question

This question asks whether mathematically correct textbook RSA is also secure. RSA can correctly encrypt and decrypt a small integer while still leaking structure, permitting chosen-ciphertext attacks in poorly designed protocols, or exposing data when parameters are weak. Padding and protocol design are essential.

Core concepts and technical explanation

RSA key generation selects distinct large primes p and q, calculates n=pq and φ(n)=(p−1)(q−1), chooses e relatively prime to φ(n), and computes d=e^−1 mod φ(n). The public key (n,e) can be distributed. The private key containing d and the prime factors must be protected.

Textbook RSA encryption is C=M^e mod n. Decryption is M=C^d mod n. The message representative must satisfy 0≤M<n. For practical byte strings, a defined encoding and padding scheme is required; one cannot simply interpret an arbitrary long file as a single integer and apply textbook RSA.

Correctness follows from the relationship ed≡1 mod φ(n), under the RSA number-theoretic assumptions and appropriate message domain. The example is useful for learning, but small p and q make factoring trivial. A production key requires large, randomly generated primes and an approved modulus size.

Textbook RSA is deterministic: the same message and public key always produce the same ciphertext. Repeated messages therefore produce repeated ciphertexts. Small-message and small-exponent situations can also be dangerous, especially when an attacker has multiple related ciphertexts or can exploit a protocol that exposes decryption behaviour.

Factoring n reveals p and q. Once p and q are known, an attacker computes φ(n), then obtains d as the inverse of e modulo φ(n). Therefore the security of RSA depends on the difficulty of factoring a properly generated large modulus, as well as safe encoding, implementation and key management.

OAEP (Optimal Asymmetric Encryption Padding) randomises RSA encryption using a structured encoding based on a mask-generation function and a hash. It prevents the simple deterministic mapping of plaintext to ciphertext and is the standard padding approach for RSA encryption in modern protocols. OAEP must be implemented using a vetted library with consistent error handling.

Design, worked reasoning and evidence

Hand-checkable RSA example: use p=5, q=11, n=55, φ(n)=40, e=3 and d=27. Encrypt M=7: C=7^3 mod 55=343 mod 55=13. Decrypt: M=13^27 mod 55. Square-and-multiply gives 7, confirming recovery. Because n=55 is tiny, this is only a classroom demonstration.

Repeated-message test: encrypt M=7 twice using textbook RSA with the same public key. Both outputs are 13. This demonstrates determinism. Repeat with OAEP using a secure library and fresh random padding; the same plaintext should normally produce different ciphertexts. Record the library, parameters and randomness behaviour.

Boundary tests should include M=0, M=1, M=n−1 and invalid M≥n. The implementation should reject invalid representatives rather than silently wrap them. For actual text, use the library's OAEP interface rather than manually designing byte encodings.

Factoring demonstration: factor the toy modulus n=55 as 5×11. Compute φ(n)=40. Given e=3, calculate d=27. This demonstrates the dependency chain: factor n → compute φ(n) → recover d. Trial division and Fermat factorisation can be demonstrated on small moduli; Fermat is particularly effective when p and q are close because n can be written as a difference of squares.

Fermat factorisation starts with a=ceil(sqrt(n)) and searches for a^2−n=b^2. Then n=(a−b)(a+b). Use a modulus formed from two close toy primes and compare the number of iterations with a modulus containing a small factor. State that the observed toy result depends on the selected values and does not establish a general production attack.

OAEP evaluation: the randomised encoding makes encryption probabilistic, reduces structural leakage and is designed to prevent classes of attacks against raw RSA. It does not protect a compromised private key, fix a weak random generator, authenticate the sender by itself or replace protocol-level validation. RSA encryption alone does not prove who sent a message.

Evaluation, limitations and deployment implications

Key length is only one factor. If p and q are generated predictably, if the random generator repeats, if the private key is stored in a readable file, or if a side-channel leaks operations, an attacker may succeed without factoring a properly generated modulus.

RSA is computationally expensive compared with symmetric encryption and is not suitable for directly encrypting large clinical files. Use it for short key material or key establishment in a hybrid design. Large files should be encrypted using an authenticated symmetric cipher.

Padding errors should not reveal whether a ciphertext failed at one specific internal validation step. Detailed decryption error differences can become an oracle. Production implementations should use vetted libraries and protocol-defined error handling rather than custom RSA code.

Question conclusion

Q3 shows that textbook RSA's correct decryption property is not sufficient evidence of security. Randomised OAEP padding, strong key generation, careful error handling and hybrid encryption are required, and RSA should not be used as a bulk-file cipher.

Test	Prediction	Evidence to record	Security meaning
Same message twice with textbook RSA	Same ciphertext	Two ciphertext values	Determinism reveals equality.
Same message twice with OAEP	Different ciphertexts expected	Two ciphertext values and library parameters	Randomisation reduces structural leakage.
Factor toy modulus	Factors found quickly	p, q, φ(n), recovered d	Small keys are not secure.
M outside [0,n−1]	Reject input	Error / validation result	Avoid invalid representative handling.
Tampered ciphertext	Decryption/validation failure	Observed response	Avoid informative error oracles.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Objective and interpretation of the question

Diffie–Hellman allows two parties to derive a common secret over a public channel without transmitting that secret directly. However, unauthenticated Diffie–Hellman does not establish who is at the other end. An active attacker can conduct two separate exchanges and relay traffic. Authentication and operational key management are therefore essential.

Core concepts and technical explanation

Public parameters include a large prime p and a suitable generator g. Clinic A chooses private exponent a and sends A=g^a mod p. Clinic B chooses private exponent b and sends B=g^b mod p. A calculates K=B^a mod p and B calculates K=A^b mod p. Both obtain g^(ab) mod p.

The equality follows because (g^a)^b ≡ g^(ab) ≡ (g^b)^a (mod p). The private exponents remain secret; the public values can be observed. The security intuition is that computing the shared secret from public values requires solving a discrete logarithm problem in a suitable group.

Small toy groups are intentionally weak. A brute-force discrete-log demonstration can enumerate candidate exponents until g^x mod p equals the public value. As p and group parameters increase, the search becomes infeasible for classical attackers when approved parameters and implementations are used.

A man-in-the-middle attacker can intercept A's public value and send the attacker's own public value to B, then intercept B's value and send another attacker-controlled value to A. The attacker now shares one key with A and a different key with B, and can decrypt and re-encrypt relayed traffic.

Digital signatures and certificates authenticate the public values. A certificate binds a public key to an identity through a trusted certificate authority. The peers validate the certificate chain, name, validity period, key usage and revocation status, then verify a signature over the exchange transcript. Transcript binding prevents an attacker from substituting unauthenticated values.

Design, worked reasoning and evidence

Toy example: let p=23 and g=5. Clinic A chooses a=6 and sends A=5^6 mod 23=8. Clinic B chooses b=15 and sends B=5^15 mod 23=19. A calculates 19^6 mod 23=2; B calculates 8^15 mod 23=2. Both obtain shared value 2. This small example is for hand calculation only.

Message sequence: (1) A and B agree on validated public group parameters. (2) A sends its ephemeral public value and identity proof. (3) B sends its ephemeral public value and identity proof. (4) Each verifies the other's certificate/signature and the complete transcript. (5) Both derive the shared secret. (6) A key-derivation function derives separate encryption and authentication keys. (7) The application uses the derived traffic keys for a limited session.

MITM demonstration: attacker Mallory chooses x and y. Mallory sends g^x to B while pretending to be A, and sends g^y to A while pretending to be B. A derives a key with Mallory; B derives a different key with Mallory. Mallory can read a message under the first key and forward a modified or

unchanged message under the second key. The attack succeeds because neither endpoint authenticated the exchange.

Authenticated exchange defence: each endpoint signs the ephemeral key share and transcript with its long-term signing key. The receiver validates the certificate identity and signature before accepting the share. If Mallory substitutes a value, the signature no longer verifies. The transcript should include protocol version, identities, algorithms and both public shares to prevent cross-protocol or identity-confusion errors.

Key management lifecycle: generate keys using a cryptographically secure random generator; store private keys in protected hardware or a restricted keystore; distribute certificates through an authenticated process; rotate keys on schedule and after compromise; revoke compromised certificates; keep short-lived session keys; separate encryption and signing keys; maintain audit logs; and define recovery procedures for lost or compromised clinic devices.

Evaluation, limitations and deployment implications

Diffie–Hellman by itself provides key agreement, not identity, not message integrity and not non-repudiation. A secure design must authenticate the exchange and use a key-derivation function rather than treating the raw group element as an application key.

Long-term static private keys and ephemeral session keys have different roles. Ephemeral keys can provide forward secrecy when properly used and erased, so later compromise of a long-term authentication key does not automatically reveal past session traffic. This depends on protocol construction and secure deletion.

For 25 or many more clinics, a certificate-based identity system scales better than manually sharing one common secret across all locations. A central certificate authority or managed PKI can issue individual device and service certificates, track revocation and support replacement. It also creates operational responsibilities: protect the CA, renew certificates and monitor expiry.

Question conclusion

Q4 demonstrates that Diffie–Hellman solves the shared-secret establishment problem but not the authentication problem. The recommended design uses ephemeral key agreement authenticated by certificates/signatures, a key-derivation function and short-lived session keys supported by a managed lifecycle.

Lifecycle stage	Required control	Healthcare example
Generation	Secure random source; approved parameters	Create separate clinic device and service key pairs.
Distribution	Authenticated certificates / protected provisioning	Issue each clinic a unique certificate.
Storage	Hardware-backed or access-controlled keystore	Do not store private keys in plain configuration files.
Use	Separate signing and key-agreement purposes	Do not reuse one key for unrelated operations.

Rotation	Defined validity and renewal policy	Renew before expiry; rotate session keys frequently.
Revocation	Rapid invalidation and status checking	Disable a lost clinic device certificate.
Recovery	Incident procedure and re-enrolment	Replace keys after compromise; investigate exposure.
Audit	Record issuance, use and revocation events	Retain logs without exposing secret key material.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Objective and interpretation of the question

Elliptic-curve cryptography provides public-key operations based on the algebra of points on an elliptic curve over a finite field. It can provide comparable classical security with substantially smaller public-key sizes than traditional RSA parameters, which is valuable for constrained clinic devices and low-bandwidth links.

Core concepts and technical explanation

An elliptic curve over a prime field commonly has an equation $y^2 \equiv x^3 + ax + b \pmod{p}$, subject to a non-singularity condition. The points on the curve, together with a point at infinity, form a mathematical group. The group operation is point addition; repeated addition of a point is scalar multiplication.

ECC key generation selects a private scalar d and computes public point Q=dG, where G is a designated base point. The private scalar must be random and protected. The public point must be validated according to the curve and protocol requirements.

In an elliptic-curve Diffie–Hellman exchange, A selects private scalar a and sends A=aG. B selects b and sends B=bG. A computes aB=a(bG)=abG; B computes bA=b(aG)=abG. Both derive the same shared point and then use a key-derivation function to obtain symmetric keys.

RSA security is associated with the difficulty of factoring a large composite modulus. Finite-field Diffie–Hellman uses the discrete-logarithm problem in a finite field. ECC uses the elliptic-curve discrete-logarithm problem. These are different mathematical problems, and key-size comparisons should be based on approved security-strength guidance rather than comparing raw bit counts alone.

ECC's smaller public keys reduce certificate size, network overhead and storage requirements. This can be useful for devices with limited memory, bandwidth or processing capacity. The advantage depends on the selected curve, implementation and hardware; it does not mean ECC is automatically faster in every environment.

Design, worked reasoning and evidence

Toy ECC arithmetic should use a small prime p, a nonsingular curve and a base point whose order is known. Enumerate the curve points, demonstrate point addition and double a point repeatedly to calculate scalar multiplication. Choose private scalars a and b, calculate public points aG and bG, and verify that a(bG)=b(aG). Since toy fields are tiny, the private scalar can be recovered by enumeration and the example is not secure.

Example experiment record: state p, a, b, base point G, the point order, private scalars, public points and shared point. Include intermediate point additions so the result is reproducible. If a point at infinity occurs, handle it according to the group law rather than treating it as an ordinary coordinate pair.

Comparison should cover: RSA key size and modular exponentiation; finite-field DH parameter and exponent sizes; ECC curve and scalar sizes; public-key/certificate bandwidth; operation time on the target device; availability of reviewed libraries; and long-term support. The exact figures depend on the security level and standard selected, so use a current standards table when preparing a real engineering specification.

Evaluation, limitations and deployment implications

ECC implementation is sensitive to poor randomness, invalid-curve inputs, side channels, incorrect point validation and private-key leakage. Use named, approved curves and constant-time implementations from vetted libraries. Do not invent a curve for deployment.

ECC does not solve authentication by itself. A public key must still be bound to an identity, normally using certificates or another trusted provisioning mechanism. An unauthenticated ECDH exchange remains vulnerable to a man-in-the-middle attack.

For constrained devices, an authenticated ephemeral elliptic-curve key agreement is often an efficient choice, while a modern authenticated symmetric cipher protects the file. The exact choice should be validated against device capability, interoperability, compliance and the organisation's migration plan.

Question conclusion

Q5 supports ECC as a strong candidate for constrained clinic devices because of its smaller public-key material. A practical recommendation is authenticated ephemeral ECDH using approved parameters, certificate validation and a secure key-derivation function, with symmetric authenticated encryption for bulk records.

Criterion	RSA	Finite-field DH	ECC / ECDH
Main mathematical problem	Integer factorisation	Finite-field discrete logarithm	Elliptic-curve discrete logarithm
Typical role	Encryption of short key material; signatures	Key agreement	Key agreement and signatures (scheme-dependent)
Public material	Large modulus and exponent	Group parameters and public value	Curve identifier and compact point

Constrained devices	Larger key/certificate overhead	Can be relatively heavy	Often compact and efficient
Authentication	Certificate/signature needed for identity	Authentication required	Authentication required
Important risks	Weak primes, padding misuse, side channels	Weak groups, small/subgroup issues, MITM	Invalid points, weak randomness, side channels

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Objective and interpretation of the question

The healthcare network exchanges large diagnostic reports and clinical files. Public-key encryption of every data block is inefficient. A hybrid design uses symmetric encryption for the payload and public-key cryptography for key establishment or key protection. Digital signatures provide sender authentication and integrity when verification keys are trusted.

Core concepts and technical explanation

A digital signature normally hashes the message and signs the digest with the sender's private signing key. The receiver recomputes the hash and verifies the signature using the sender's public key. If the message changes, the digest changes and verification fails. The signature also provides evidence that the holder of the private signing key approved the signed data, subject to correct identity validation and key custody.

RSA signatures, DSA/DSS and ECDSA are different signature families. RSA signatures rely on RSA arithmetic and a defined encoding such as PSS. DSA and ECDSA rely on discrete-logarithm assumptions and require careful nonce generation. Reusing or biasing a signing nonce can expose the private signing key. The signature algorithm, hash and encoding must be selected as a compatible approved suite.

Hybrid encryption generates a fresh random symmetric session key and encrypts the large file using an authenticated symmetric algorithm. The session key is then protected for the recipient using public-key encryption or established through authenticated key agreement. The receiver obtains the session key and decrypts the payload. This reduces expensive public-key operations to a small amount of key material rather than every file block.

An authenticated hybrid protocol must bind the encrypted payload, session key context, sender identity, recipient identity, algorithm identifiers, nonces and protocol transcript. A digital signature can authenticate the sender and protect metadata. An AEAD tag detects ciphertext modification. A signature and an AEAD tag serve related but different purposes.

Quantum key distribution uses quantum states to establish or distribute key material and can reveal certain eavesdropping attempts through disturbance of the quantum states. It requires specialised physical infrastructure and authenticated classical communication. It does not automatically

authenticate endpoints, secure endpoints, protect stored data, replace key management or provide a universal replacement for conventional cryptography.

A sufficiently capable fault-tolerant quantum computer running Shor's algorithm would threaten widely deployed RSA and elliptic-curve public-key systems. Symmetric cryptography is affected differently; larger key sizes can mitigate known quantum search advantages. A migration plan should inventory cryptographic dependencies, adopt crypto-agility, follow post-quantum standards and support hybrid classical/post-quantum key establishment during transition.

Design, worked reasoning and evidence

Signature flow: sender serialises the report and relevant metadata; computes a cryptographic hash; signs the digest using a private signing key; sends the report, signature and certificate. Receiver validates the certificate chain and identity, checks revocation/validity, recomputes the hash and verifies the signature. Any mismatch causes rejection and an audit event.

Tamper test 1 – file: change one byte of the received report but retain the original signature. Verification should fail because the recomputed digest differs. Tamper test 2 – signature: flip one signature bit; verification should fail. Tamper test 3 – session key: alter or replace the wrapped session key; key unwrapping or authenticated payload verification should fail. Record expected result, observed result and error handling.

Hybrid workflow: (1) generate a fresh random content-encryption key (CEK); (2) generate a unique nonce; (3) encrypt the file using an authenticated symmetric cipher and include associated metadata; (4) establish or wrap the CEK using the recipient's public-key mechanism; (5) sign the package manifest and relevant metadata; (6) transmit ciphertext, nonce, wrapped CEK, signature and certificate; (7) receiver validates identity and signature, obtains CEK, checks AEAD tag and releases plaintext only after all checks pass.

Operation-count comparison: direct RSA on a large file requires splitting or encoding many blocks and performing public-key operations repeatedly, which is inefficient and constrained by RSA's message-size limits. Hybrid encryption uses a small number of public-key operations for the CEK and efficient symmetric operations for every data block. Record actual calls and byte counts rather than claiming a universal speed ratio.

Quantum readiness table: inventory RSA, DH and ECC uses; identify certificates, VPNs, TLS endpoints, medical devices, backup archives and signed records; prioritise long-lived confidential data that could be harvested now and decrypted later; test post-quantum key establishment and signatures; deploy hybrid negotiation where standards permit; maintain algorithm agility and a documented deprecation schedule.

Evaluation, limitations and deployment implications

Digital signatures do not encrypt the report. Anyone with access to the transmitted report may read it unless encryption is also applied. Conversely, encryption does not automatically identify the sender. Confidentiality, integrity and authentication must be addressed explicitly.

Key custody is central to signature trust. If a clinic signing key is shared by many staff members, attribution is weak. Use individual or device identities, protected private keys, revocation,

timestamping where required and audit records. A compromised signing key may allow an attacker to create apparently valid future signatures until revocation and incident handling take effect.

Quantum key distribution is not a substitute for ordinary cryptographic management. It does not remove the need for endpoint security, authenticated classical channels, identity governance, access control, secure software, backup protection or symmetric data encryption. It is a specialised option where infrastructure and threat model justify it.

Post-quantum migration must be planned before a cryptographically relevant quantum computer exists. Systems that retain sensitive patient data for many years face a 'harvest now, decrypt later' concern. Migration should begin with inventory, data lifetime classification, protocol agility, vendor testing and staged deployment.

Question conclusion

Q6 recommends a hybrid architecture: authenticated key agreement or public-key protection for a fresh session key, authenticated symmetric encryption for large files, and a digital signature for sender authenticity and integrity. Quantum readiness is handled through inventory and staged migration to standardised post-quantum mechanisms, not by assuming QKD replaces all security controls.

Requirement	Mechanism	What it contributes	What it does not solve alone
Large-file confidentiality	AEAD symmetric encryption	Efficient bulk encryption and tag verification	Sender identity unless key is identity-bound.
Session-key establishment	Authenticated ECDH / approved key transport	Shared or protected short-lived key	Trust in identity without certificate/signature validation.
Sender authenticity	Digital signature + certificate	Verifiable origin and integrity	Confidentiality.
Tamper detection	AEAD tag and/or signature	Detects modification	Availability or endpoint compromise.
Quantum migration	Post-quantum / hybrid protocol plan	Reduces future public-key exposure	Does not secure weak endpoints or poor key management.

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Objective and interpretation of the question

This design challenge requires the student to identify a meaningful weakness or bottleneck, define a measurable objective, redesign the system, integrate quantum readiness, predict the result and validate

the revised architecture. The answer must be based on evidence from Q1–Q6 rather than simply listing more algorithms.

Core concepts and technical explanation

A realistic bottleneck is the use of unauthenticated key exchange combined with long-lived shared secrets across many clinics. As the number of clinics grows, one shared key increases the impact of compromise. If an attacker can perform a man-in-the-middle attack during key establishment, the attacker may establish separate keys with both endpoints and read or modify traffic. A second bottleneck is attempting to use public-key operations for bulk files.

Another important weakness is poor nonce or key reuse. In CTR or stream encryption, reusing a nonce with the same key repeats the keystream. Repeated ciphertext patterns can expose relationships between messages. For AEAD modes, nonce reuse may also compromise authentication. The redesigned system must ensure uniqueness and have a clear key/nonce lifecycle.

The measurable design objective can be stated as: (1) no clinic session is accepted unless the peer identity and key-exchange transcript are authenticated; (2) every bulk file uses a fresh session key and unique nonce; (3) public-key operations are limited to session establishment or key wrapping; (4) tampered files are rejected before clinical use; and (5) a lost clinic credential can be revoked without changing every other clinic's identity.

Revised architecture: each clinic has a unique device/service identity and certificate; authenticated ephemeral ECDH establishes a fresh shared secret; a key-derivation function produces traffic keys; an AEAD cipher encrypts the file; a digital signature protects the sender identity and manifest where required; session keys expire and are erased; revocation blocks compromised devices; logs record security events; and a crypto-agility layer supports future post-quantum suites.

Quantum migration should distinguish key establishment and signatures. RSA and ECC key establishment/signatures are threatened by a sufficiently capable quantum computer. The network should test and deploy standardised post-quantum algorithms as supported by its ecosystem, potentially using hybrid classical/post-quantum key establishment during transition. QKD may be considered only for specialised links with suitable fibre/optical infrastructure and a clear operational justification.

Design, worked reasoning and evidence

Prediction before implementation: authenticated key exchange should prevent the toy MITM substitution because substituted public values will not verify against the signed transcript. Hybrid encryption should reduce public-key operations from repeated per-block processing to a small fixed number per file. AEAD tamper tests should reject altered ciphertext or metadata. Per-clinic credentials should limit the blast radius of a single clinic compromise.

Validation plan: create two clinic identities and one test server identity; issue test certificates; run authenticated key agreement; derive a session key; encrypt a 1 KB, 1 MB and 10 MB fictional file; transmit and decrypt; alter one ciphertext byte and confirm tag failure; alter the signed manifest and confirm signature failure; revoke one clinic certificate and confirm a new connection is rejected; record operation counts and execution times.

Comparison table should include original design and redesigned design across authentication, MITM resistance, bulk-file operations, key scope, nonce policy, revocation, tamper detection, quantum migration and operational complexity. State the exact test parameters and avoid inventing measured times. If a library is used, include its name/version and configuration.

Operation-count interpretation: direct public-key encryption of large payloads scales with the number of encoded blocks and is constrained by the maximum input size of the public-key scheme. Hybrid design performs a small number of public-key operations to establish or protect the session key, while symmetric encryption scales linearly with file size and is optimised for bulk data. This is a structural performance improvement even before measuring wall-clock time.

Key-management event comparison: a network-wide shared secret requires coordinated replacement when compromised and makes every site a high-value target. Unique clinic credentials allow targeted revocation and re-enrolment. This increases certificate issuance and monitoring work but reduces the compromise radius and supports accountability.

Evaluation, limitations and deployment implications

The redesign does not eliminate all risks. Malware on a clinic workstation can read plaintext before encryption or after decryption. A compromised signing key can create fraudulent records. Poor certificate validation can reintroduce MITM. Denial of service can block availability. Insider misuse requires least privilege, monitoring, separation of duties and incident response.

Large-file encryption still requires sufficient memory, storage, bandwidth and reliable recovery. Key rotation must not make archived clinical records permanently unreadable; a controlled key-escrow or recovery policy may be needed, but it must not create an unrestricted decryption backdoor. Backups and audit logs must be protected as carefully as live records.

Deployment should be staged: inventory devices and protocols; pilot with a small number of clinics; test certificate provisioning and revocation; train staff; monitor failed authentication and nonce errors; expand gradually; and retain a documented rollback plan. Low-resource sites need automated certificate renewal and minimal manual key handling.

Quantum migration adds complexity and may increase handshake size or computational load. The network should test device compatibility and measure performance before deployment. Do not claim that a quantum algorithm or QKD automatically solves identity, integrity, endpoint or governance problems.

Question conclusion

Q7's final design is authenticated ephemeral key agreement, per-clinic identities, derived short-lived session keys, authenticated symmetric encryption for bulk records, signatures for important clinical messages, revocation and a planned post-quantum migration. It addresses the identified MITM and scaling weaknesses while acknowledging implementation and operational risks.

Design property	Original weak design	Revised design	Validation evidence
Identity	Unauthenticated DH or shared key	Certificate-authenticated ephemeral exchange	MITM substitution rejected.

Bulk data	Public-key operations repeated or unauthenticated mode	AEAD symmetric encryption	Operation counts and file tests.
Key scope	One long-lived shared key	Per-clinic identity; per-session traffic key	Revoke one clinic independently.
Nonce handling	Potential reuse	Unique nonce per key; tracked generation	Duplicate nonce test rejected.
Integrity	Encryption assumed to imply integrity	AEAD tag and signatures	Byte-tampering test fails.
Quantum planning	No inventory or transition	Crypto-agility and post-quantum pilot	Compatibility and migration test.

Appendix A – Student implementation plan and pseudocode

A1. Euclid and extended Euclid

Pseudocode: while b ≠ 0, set (a,b)=(b,a mod b). For the extended form, maintain the coefficient pairs for a and b and update them with the quotient at each division. At completion, return gcd and coefficients x,y. Check ax+by=gcd(a,b).

A2. Square-and-multiply

Set result=1 and base=a mod n. While exponent>0: if exponent is odd, set result=(result×base) mod n; set base=(base×base) mod n; set exponent=floor(exponent/2). Return result. Test exponent 0, exponent 1 and values larger than n.

A3. RSA toy implementation

Generate only small demonstration keys by hand. Compute n, φ(n), choose e with gcd(e,φ(n))=1, compute d with extended Euclid, encrypt using modular exponentiation and decrypt using the same function. Add validation for invalid message representatives. Never use the toy key generator for real information.

A4. Diffie–Hellman toy implementation

Select a small prime p and base g, choose private exponents a and b, compute public values g^a mod p and g^b mod p, then calculate both shared values. Assert equality. Add an explicit MITM simulation with two attacker exponents to demonstrate that equality between the two honest endpoints is not guaranteed without authentication.

A5. ECC toy implementation

Represent each finite curve point as a coordinate pair and the point at infinity as a special value. Implement point addition for distinct points and point doubling for equal points, including modular inverses. Implement scalar multiplication using double-and-add. Validate that the chosen base point lies on the curve and show that the two parties derive the same shared point.

Appendix B – Experimental evidence recording templates

Experiment	Input sizes / cases	Record	Interpretation
Symmetric algorithms	Same test file for all algorithms	Algorithm, key/parameters, ciphertext size, setup and run time	Efficiency and compatibility.
Modes	Repeated blocks; changed IV/nonce; changed plaintext	Ciphertext pattern, decryption result, tag result	Pattern leakage and nonce safety.
Number theory	Coprime and non-coprime values	Loop counts, divisions, inverse result	Efficiency and inverse existence.
Primality	Prime, composite and Carmichael candidates	Bases, rounds, accept/reject, false acceptance	Limits of each test.
RSA	Toy key, repeated plaintext, invalid boundary	Ciphertext, decrypted message, factorisation steps	Determinism and weak-key risk.
DH	Toy p values and MITM test	Public values, shared secret, attack success	Authentication requirement.
Hybrid	1 KB, 1 MB, 10 MB test files	Public-key calls, symmetric calls, total time	Bulk-data suitability.
Tamper	Flip file, signature and key-package bits	Verification / authentication result	Integrity behaviour.

Appendix C – Suggested source-code test cases

- Test GCD with coprime inputs, shared factors, zero second operand and reversed operands.
- Test modular inverse with a valid coprime pair and an invalid non-coprime pair.
- Test modular exponentiation with exponent zero, exponent one, a base larger than the modulus and a large exponent.
- Test primality methods with small primes, small composites and Carmichael numbers 561, 1105 and 1729.
- Test RSA with valid toy keys, invalid message representatives, repeated plaintext and a factored weak modulus.
- Test Diffie–Hellman with two honest parties, wrong public values and a simulated active attacker.
- Test ECC point membership, point addition, point doubling, scalar multiplication and shared-point equality.

- Test signatures with unchanged content, modified content, modified signature and an untrusted certificate.
- Test hybrid encryption with empty file, small file, large file, wrong key, wrong nonce and modified ciphertext.
- Test revocation by disabling one clinic credential and confirming that other clinic credentials continue to work.

Appendix D – Security interpretation and viva preparation

Why is a public network not automatically unsafe for cryptography?

Public networks can carry protected traffic when the cryptographic protocol authenticates endpoints, establishes fresh keys, encrypts data and detects modification. The network should still be treated as observable and attacker-controlled.

Why is encryption not the same as authentication?

Encryption hides content. Without authentication, an attacker may impersonate a peer or modify malleable ciphertext. Authentication binds a key or message to an identity and detects unauthorised changes.

Why not encrypt a large file directly with RSA?

RSA is designed for short encoded messages and is computationally expensive. Hybrid encryption uses symmetric encryption for the file and public-key cryptography only for the session key.

Why must a nonce not repeat?

Modes such as CTR and common AEAD constructions rely on unique nonce values for each encryption under a given key. Reuse can repeat keystream material and may undermine confidentiality and authentication.

Why is certificate validation necessary in Diffie–Hellman?

Without authentication, an attacker can replace public values and create separate shared secrets with both parties. A validated certificate and signature bind the exchange to the expected identity.

What does quantum readiness mean?

It means inventorying vulnerable cryptographic uses, planning algorithm agility, testing standardised post-quantum methods and migrating before long-lived confidential data becomes exposed to future decryption.

Final integrated conclusion

The healthcare network should not rely on a single algorithm or on encryption alone. Symmetric encryption is the efficient choice for large patient files, but it must be authenticated and use correctly managed keys and nonces. Public-key cryptography supports secure key establishment and identity binding. RSA remains important for understanding public-key encryption and signatures, while authenticated ephemeral elliptic-curve key agreement can offer compact public-key material for constrained devices. Digital signatures help verify important clinical messages, and certificate

management provides scalable identity and revocation. The recommended architecture is hybrid, authenticated, auditable and crypto-agile. All toy calculations in this report are learning demonstrations; real patient data requires approved modern cryptographic standards and professionally managed implementations.

References and further reading

- Course assignment sheet: Assignment 2 – CO2, Cryptography, supplied by the course faculty. The question order and assessment requirements in this report follow that sheet.
- National Institute of Standards and Technology (NIST), FIPS 197, Advanced Encryption Standard (AES).
- NIST, SP 800-56A, Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography.
- NIST, SP 800-57 Part 1, Recommendation for Key Management.
- NIST, SP 800-38A, Recommendation for Block Cipher Modes of Operation.
- NIST, SP 800-56C, Recommendation for Key-Derivation Methods in Key-Establishment Schemes.
- NIST, FIPS 186-5, Digital Signature Standard.
- NIST, SP 800-208 and current NIST post-quantum cryptography publications for migration context.
- Any software library documentation actually used in the student's implementation; record the library name, version and algorithm parameters.